

Need azure account for all the videos of this week

Session 3,4,5:(dlt-demo-1)

Python equivalent codes:

Step 1: Load the data from landing zone and move it to bronze layer

```
CREATE STREAMING TABLE orders_bronze
as
SELECT *,
input_file_name() as filename,
current_timestamp() as load_time
FROM cloud_files('/mnt/landing/orders', 'csv',
map("cloudFiles.inferColumnTypes", "True"));

CREATE STREAMING TABLE customers_bronze
as
SELECT *,
input_file_name() as filename,
current_timestamp() as load_time
FROM cloud_files('/mnt/landing/customers', 'csv',
map("cloudFiles.inferColumnTypes", "True"));
```

Python code:

```
from pyspark.sql.functions import input_file_name, current_timestamp
```

```
orders_bronze_df = (
    spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("cloudFiles.inferColumnTypes", "True")
        .load("/mnt/landing/orders")
        .withColumn("filename", input_file_name())
        .withColumn("load_time", current_timestamp())
)
```

```
orders_bronze_df.writeStream \
    .format("delta") \
    .option("checkpointLocation", "/mnt/checkpoints/orders_bronze") \
    .trigger(availableNow=True) \
    .toTable("orders_bronze")
```

```

customers_bronze_df = (
    spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("cloudFiles.inferColumnTypes", "True")
        .load("/mnt/landing/customers")
        .withColumn("filename", input_file_name())
        .withColumn("load_time", current_timestamp())
)

customers_bronze_df.writeStream \
    .format("delta") \
    .option("checkpointLocation", "/mnt/checkpoints/customers_bronze") \
    .trigger(availableNow=True) \
    .toTable("customers_bronze")

```

Step 2: Take the data from bronze, clean it and load it to silver cleaned

```

CREATE STREAMING TABLE orders_silver_cleaned (
    CONSTRAINT valid_order EXPECT (order_id is NOT NULL) ON VIOLATION DROP ROW,
    CONSTRAINT valid_customer EXPECT (customer_id is NOT NULL) ON VIOLATION DROP
ROW
) as
SELECT orderid as order_id,
orderdate as order_date,
customerid as customer_id,
totalamount as total_amount,
status,
filename as file_name,
load_time
FROM STREAM(live.orders_bronze);

CREATE STREAMING TABLE customers_silver_cleaned(
    CONSTRAINT valid_customer EXPECT (customer_id is NOT NULL) ON VIOLATION DROP
ROW
) as
SELECT customerid as customer_id,
customername as customer_name,
address as city,
dateofbirth as dob,

```

```
registrationdate as customer_since,  
filename as file_name,  
load_time  
from STREAM(live.customers_bronze);
```

Python code:

```
import dlt  
from pyspark.sql.functions import col  
  
@dlt.table(  
    name="orders_silver_cleaned",  
    comment="Cleaned orders data with valid order_id and customer_id",  
)  
@dlt.expect_or_drop("valid_order", "order_id IS NOT NULL")  
@dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL")  
def orders_silver_cleaned():  
    return (  
        dlt.read_stream("orders_bronze")  
        .select(  
            col("orderid").alias("order_id"),  
            col("orderdate").alias("order_date"),  
            col("customerid").alias("customer_id"),  
            col("totalamount").alias("total_amount"),  
            col("status"),  
            col("filename").alias("file_name"),  
            col("load_time"),  
        )  
    )  
  
@dlt.table(  
    name="customers_silver_cleaned",  
    comment="Cleaned customers data with valid customer_id",  
)  
@dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL")  
def customers_silver_cleaned():  
    return (  
        dlt.read_stream("customers_bronze")  
        .select(  
            col("customerid").alias("customer_id"),  
            col("customername").alias("customer_name"),  
            col("address").alias("city"),  
            col("dateofbirth").alias("dob"),  
            col("registrationdate").alias("customer_since"),
```

```
        col("filename").alias("file_name"),
        col("load_time"),
    )
)
```

Step 3: Customers- Take the data from Silver Cleaned, handle duplicates (SCD2) and load in Silver layer

```
CREATE STREAMING TABLE customers_silver;

APPLY CHANGES into live.customers_silver
from STREAM(live.customers_silver_cleaned)
keys(customer_id)
sequence by load_time
stored as scd type 2;
```

Python Code:

```
import dlt

@dlt.table(
    name="customers_silver",
    comment="SCD Type 2 dimension table for customers"
)
def customers_silver():
    return dlt.read("customers_silver_cleaned") # empty base definition

dlt.apply_changes(
    target="customers_silver",
    source="customers_silver_cleaned",
    keys=["customer_id"],
    sequence_by="load_time",
    stored_as_scd_type="2"
)
```

step 4: Orders- Take the data from silver cleaned, handle duplicates (Merge) and load in silver layer

```
CREATE STREAMING TABLE orders_silver;

APPLY CHANGES into live.orders_silver
from STREAM(live.orders_silver_cleaned)
```

```
keys(order_id)
sequence by load_time;
```

Python Code:

```
import dlt
```

```
@dlt.table(
    name="orders_silver",
    comment="Type 1 table containing latest order records"
)
def orders_silver():
    # Empty definition - target for apply_changes
    return dlt.read("orders_silver_cleaned")

dlt.apply_changes(
    target="orders_silver",
    source="orders_silver_cleaned",
    keys=["order_id"],
    sequence_by="load_time"
)
```

step 5: Gold Layer- Take the data from Silver and perform aggregations and load to gold layer.

```
CREATE MATERIALIZED VIEW city_wise_sales_gold
as
SELECT city, sum(total_amount) as total_sales
from live.orders_silver o
join live.customers_silver c
on o.customer_id = c.customer_id
group by city;
```

Python Code:

```
import dlt
from pyspark.sql.functions import sum as _sum
```

```
@dlt.table(
    name="city_wise_sales_gold",
    comment="Aggregated total sales by city"
)
def city_wise_sales_gold():
    orders_df = dlt.read("orders_silver")
    customers_df = dlt.read("customers_silver")
```

```

return (
    orders_df.join(customers_df, on="customer_id", how="inner")
    .groupBy("city")
    .agg(_sum("total_amount").alias("total_sales"))
)

```

Session 6: (dlt-demo-2)

Step 1: Load the data from landing zone and move it to bronze layer

```

CREATE STREAMING TABLE orders_bronze
as
SELECT *,
_metadata.file_name as filename,
current_timestamp() as load_time
FROM
cloud_files('abfss://retail@ttmystorageaccount001.dfs.core.windows.net/input/orders', 'csv', map("cloudFiles.inferColumnTypes", "True"));

CREATE STREAMING TABLE customers_bronze
as
SELECT *,
_metadata.file_name as filename,
current_timestamp() as load_time
FROM
cloud_files('abfss://retail@ttmystorageaccount001.dfs.core.windows.net/input/customers', 'csv', map("cloudFiles.inferColumnTypes", "True"));

```

Python Code:

```

import dlt
from pyspark.sql.functions import current_timestamp, input_file_name

@dlt.table(
    name="orders_bronze",
    comment="Raw orders data ingested from ADLS using Auto Loader"
)
def orders_bronze():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("cloudFiles.inferColumnTypes", "True")
        .load("abfss://retail@ttmystorageaccount001.dfs.core.windows.net/input/orders")
        .withColumn("filename", input_file_name())
    )

```

```

        .withColumn("load_time", current_timestamp())
    )

@dlt.table(
    name="customers_bronze",
    comment="Raw customers data ingested from ADLS using Auto Loader"
)
def customers_bronze():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("cloudFiles.inferColumnTypes", "True")
        .load("abfss://retail@ttmystorageaccount001.dfs.core.windows.net/input/customers")
        .withColumn("filename", input_file_name())
        .withColumn("load_time", current_timestamp())
    )

```

Step 2: Take the data from bronze, clean it and load it to silver cleaned

```

CREATE STREAMING TABLE orders_silver_cleaned (
    CONSTRAINT valid_order EXPECT (order_id is NOT NULL) ON VIOLATION DROP ROW,
    CONSTRAINT valid_customer EXPECT (customer_id is NOT NULL) ON VIOLATION DROP
ROW
) as
SELECT orderid as order_id,
orderdate as order_date,
customerid as customer_id,
totalamount as total_amount,
status,
filename as file_name,
load_time
FROM STREAM(orders_bronze);

CREATE STREAMING TABLE customers_silver_cleaned(
    CONSTRAINT valid_customer EXPECT (customer_id is NOT NULL) ON VIOLATION DROP
ROW
) as
SELECT customerid as customer_id,
customername as customer_name,
address as city,

```

```

dateofbirth as dob,
registrationdate as customer_since,
filename as file_name,
load_time
from STREAM(customers_bronze);

```

Python Code:

```

import dlt
from pyspark.sql.functions import col

```

```

@dlt.table(
    name="orders_silver_cleaned",
    comment="Cleansed orders data with non-null order_id and customer_id"
)
@dlt.expect_or_drop("valid_order", "order_id IS NOT NULL")
@dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL")
def orders_silver_cleaned():
    return (
        dlt.read_stream("orders_bronze")
        .select(
            col("orderid").alias("order_id"),
            col("orderdate").alias("order_date"),
            col("customerid").alias("customer_id"),
            col("totalamount").alias("total_amount"),
            col("status"),
            col("filename").alias("file_name"),
            col("load_time")
        )
    )

```

```

@dlt.table(
    name="customers_silver_cleaned",
    comment="Cleansed customers data with non-null customer_id"
)
@dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL")
def customers_silver_cleaned():
    return (
        dlt.read_stream("customers_bronze")
        .select(
            col("customerid").alias("customer_id"),
            col("customername").alias("customer_name"),
            col("address").alias("city"),
            col("dateofbirth").alias("dob"),

```



```

        col("registrationdate").alias("customer_since"),
        col("filename").alias("file_name"),
        col("load_time")
    )
)

```

Step 3: Customers - Take the data from Silver Cleaned, handle duplicates (SCD2) and load in Silver layer

```
CREATE STREAMING TABLE customers_silver;
```

```

APPLY CHANGES into customers_silver
from STREAM(customers_silver_cleaned)
keys(customer_id)
sequence by load_time
stored as scd type 2;

```

Python Code:

```
import dlt
```

```

@dlt.table(
    name="customers_silver",
    comment="SCD Type 2 table for customers"
)
def customers_silver():
    # Empty definition — this defines the target table
    return dlt.read("customers_silver_cleaned")

dlt.apply_changes(
    target="customers_silver",
    source="customers_silver_cleaned",
    keys=["customer_id"],
    sequence_by="load_time",
    stored_as_scd_type="2"
)

```

step 4: Orders - Take the data from silver cleaned, handle duplicates (Merge) and load in silver layer

```
CREATE STREAMING TABLE orders_silver;
```

```

CREATE FLOW orders_silver_flow AS AUTO CDC
into orders_silver
from STREAM(orders_silver_cleaned)

```

```
keys(order_id)
sequence by load_time;
```

Python Code:

```
import dlt
```

```
@dlt.flow(
    name="orders_silver_flow",
    comment="Automatically applies CDC updates from orders_silver_cleaned into orders_silver"
)
@dlt.auto_cdc(
    target="orders_silver",
    keys=["order_id"],
    sequence_by="load_time"
)
def orders_silver_flow():
    return dlt.read_stream("orders_silver_cleaned")
```

step 5: Gold Layer - Take the data from Silver and perform aggregations and load to gold layer.

```
CREATE MATERIALIZED VIEW city_wise_sales_gold
as
SELECT city, sum(total_amount) as total_sales
from live.orders_silver o
join live.customers_silver c
on o.customer_id = c.customer_id
group by city;
```

Python Code:

```
import dlt
```

```
from pyspark.sql.functions import sum as _sum
```

```
@dlt.table(
    name="city_wise_sales_gold",
    comment="Materialized view: total sales aggregated by city"
)
def city_wise_sales_gold():
    orders_df = dlt.read("orders_silver")
    customers_df = dlt.read("customers_silver")

    return (
        orders_df.join(customers_df, on="customer_id", how="inner")
```

```
.groupBy("city")
  .agg(_sum("total_amount").alias("total_sales"))
)
```

Session 8:

You can execute it in databricks free edition as well.

Note: If you are using the sql code then there is no need to replace the 'live' keyword.

```
CREATE OR REFRESH STREAMING TABLE baby_names_raw
COMMENT "Popular baby first names in New York. This data was ingested from the
New York State Department of Health."
AS SELECT Year, `First Name` AS First_Name, County, Sex, Count
FROM STREAM(read_files(
  '/Volumes/deltalake_catalog1/demo3_db/demo3_volume',
  format => 'csv',
  header => true,
  mode => 'FAILFAST'));
```

```
CREATE OR REFRESH MATERIALIZED VIEW baby_names_prepared(
  CONSTRAINT valid_first_name EXPECT (First_Name IS NOT NULL),
  CONSTRAINT valid_count EXPECT (Count > 0) ON VIOLATION FAIL UPDATE
)
COMMENT "New York popular baby first name data cleaned and prepared for
analysis."
AS SELECT
  Year AS Year_Of_Birth,
  First_Name,
  Count
FROM LIVE.baby_names_raw;
```

```
CREATE OR REFRESH MATERIALIZED VIEW top_baby_names_2021
COMMENT "A table summarizing counts of the top baby names for New York for
2021."
AS SELECT
  First_Name,
  SUM(Count) AS Total_Count
FROM LIVE.baby_names_prepared
WHERE Year_Of_Birth = 2021
```

```
GROUP BY First_Name
ORDER BY Total_Count DESC
LIMIT 10;
```

Python code:

```
import dlt
```

```
from pyspark.sql.functions import col
```

```
@dlt.table(
    name="raw.baby_names_raw",
    comment="Popular baby first names in New York. Raw ingested CSV data."
)
```

```
def baby_names_raw():
```

```
    return (
        spark.readStream.format("csv")
        .option("header", True)
        .option("mode", "FAILFAST")
        .load("/Volumes/deltalake_catalog1/demo3_db/demo3_volume")
        .select(
            col("Year"),
            col("First Name").alias("First_Name"),
            col("County"),
            col("Sex"),
            col("Count")
        )
    )
```

```
@dlt.view(
    name="prepared.baby_names_prepared",
    comment="Cleaned and validated baby names data.",
    constraints={
        "valid_first_name": "First_Name IS NOT NULL",
        "valid_count": "Count > 0"
    }
)
```

```
def baby_names_prepared():
```

```
    return (
        dlt.read("raw.baby_names_raw")
        .select(
            col("Year").alias("Year_Of_Birth"),
            col("First_Name"),
            col("Count")
        )
    )
```

```

@dlt.view(
    name="curated.top_baby_names_2021",
    comment="Top 10 baby names in New York for 2021."
)
def top_baby_names_2021():
    return (
        dlt.read("prepared.baby_names_prepared")
        .filter(col("Year_Of_Birth") == 2021)
        .groupBy("First_Name")
        .sum("Count")
        .withColumnRenamed("sum(Count)", "Total_Count")
        .orderBy(col("Total_Count").desc())
        .limit(10)
    )

```

Session 9:

```

CREATE STREAMING TABLE orders_bronze
comment "ingesting from landing to bronze layer"
TBLPROPERTIES ("quality" = "bronze")
AS
SELECT * ,
    _metadata.file_name as filename,
    current_timestamp() as load_time
FROM cloud_files('/Volumes/new_catalog/dlt5_schema/landing_orders', 'csv',
    map("cloudFiles.inferColumnTypes", "true"));

```

```

CREATE STREAMING TABLE customers_bronze
AS
SELECT * ,
    _metadata.file_name as filename,
    current_timestamp() as load_time
FROM cloud_files('/Volumes/new_catalog/dlt5_schema/landing_customers', 'csv',
    map("cloudFiles.inferColumnTypes", "true"));

```

```

CREATE STREAMING TABLE orders_silver_cleaned(
    constraint valid_order EXPECT (order_id is NOT NULL) ON VIOLATION DROP ROW,
    constraint valid_customer EXPECT (customer_id is NOT NULL) ON VIOLATION DROP
ROW,

```

```
    constraint valid_amount EXPECT (total_amount > 0) ON VIOLATION FAIL UPDATE
) AS SELECT orderid AS order_id, orderdate as order_date, customerid as
customer_id, totalamount as total_amount, status,
filename as file_name,
load_time
FROM STREAM(live.orders_bronze);
```

```
CREATE STREAMING TABLE customers_silver_cleaned(
    constraint valid_customer EXPECT (customer_id is NOT NULL) ON VIOLATION DROP
ROW,
    constraint customer_since EXPECT (customer_since is NOT NULL)
)
AS SELECT CUSTOMERID AS customer_id, customername as customer_name, address as
city, dateofbirth as dob, registrationdate as customer_since,
filename as file_name,
load_time
FROM STREAM(live.customers_bronze);
```

```
create streaming table customers_silver;
```

```
apply changes into live.customers_silver
from stream(live.customers_silver_cleaned)
keys(customer_id)
sequence by load_time
COLUMNS * EXCEPT
(customer_name)
stored as scd type 2;
```

```
create streaming table orders_silver;
```

```
apply changes into live.orders_silver
from stream(live.orders_silver_cleaned)
keys(order_id)
sequence by load_time;
```

```
create materialized view orders_customers_mapped(
    CONSTRAINT no_missing_records EXPECT (c.customer_id IS NOT NULL)
```

```
)  
as select o.order_id, o.order_date, o.total_amount ,c.customer_id, c.city  
from live.orders_silver o  
left outer join live.customers_silver c  
on o.customer_id = c.customer_id;
```

```
create materialized view city_wise_sales_gold  
as select city, sum(total_amount) as total_sales, count(1) as total_orders  
from live.orders_silver o  
left outer join live.customers_silver c  
on o.customer_id = c.customer_id  
group by city;
```

Session 10 and 11 - can follow the steps as shown by sir

